**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное бюджетное образовательное учреждение высшего образования

«_____»

Кафедра _____

# ЛАБОРАТОРНАЯ РАБОТА № 7

по дисциплине «Основы алгоритмизации и программирование»

Рекурсивные алгоритмы и динамическое программирование в задачах комбинаторной оптимизации

## Вариант 6

Направление: 38.03.05
«Бизнес-информатика»
Курс: 2
Выполнил: студент группы БИ 2-1
ФИО: Голдаева Валерия Валентиновна
Проверил: _____

2026

# 1. Цель работы и формулировка задания

Цель: изучить динамическое программирование как способ устранения повторных вычислений в рекурсии; сравнить наивную, мемоизированную и итеративную реализации.

Вариант 6:

| Параметр | Значение |
|---|---|
| Задача А | Биномиальный коэффициент C(n, k), n до 28 |
| Реализации | Наивная рекурсия, мемоизация, итеративная |
| Задача Б | Максимальная сумма подотрезка, 10 000 чисел |
| Доп. | Г6 — сравнение с functools.lru_cache |

# 2. Рекуррентные соотношения

## 2.1. Биномиальный коэффициент

C(n,k) = C(n−1, k−1) + C(n−1, k); C(n,0)=C(n,n)=1; 0 при k∉[0,n]. Слагаемые — выборки, где фиксированный элемент входит / не входит.

## 2.2. Максимальная сумма подотрезка

D[i] = max(a[i], D[i−1]+a[i]); ответ = max D[i]. D[i] — лучшая сумма подотрезков, оканчивающихся в i.

# 3. Дерево рекурсии C(5,2)

Наивная: 19 вызовов; мемоизация: 13. Повторяются C(3,1), C(2,1), C(2,0) и др. При мемоизации каждая пара (n,k) считается один раз.

# 4. Листинг (с пояснениями)

Полный код — lab7-variant6/main.py (комментарий к каждому действию).

## 4.1. Наивная рекурсия

```
def binom_naive(n, k):
    global calls; calls += 1
    if k &lt; 0 or k &gt; n: return 0
    if k == 0 or k == n: return 1
    return binom_naive(n-1, k-1) + binom_naive(n-1, k)
```

## 4.2. Мемоизация

```
def binom_memo(n, k, cache=None):
    global calls; calls += 1
    if cache is None: cache = {}
    if k &lt; 0 or k &gt; n: return 0
    if k == 0 or k == n: return 1
    if (n, k) in cache: return cache[(n, k)]
    cache[(n, k)] = binom_memo(n-1, k-1, cache) + binom_memo(n-1, k, cache)
    return cache[(n, k)]
```

## 4.3. Итеративная + Кадане

```
def binom_iter(n, k):
    k = min(k, n-k); res = 1
    for i in range(1, k+1):
        res = res * (n-k+i) // i
    return res

def max_subarray(arr):
    best = cur = arr[0]; best_l = best_r = cur_l = 0
    for i in range(1, len(arr)):
        if cur + arr[i] &lt; arr[i]:
            cur = arr[i]; cur_l = i
        else:
            cur += arr[i]
```

```
    if cur &gt; best:
        best, best_l, best_r = cur, cur_l, i
return best, best_l, best_r
```

# 5. Результаты измерений

## 5.1. C(n, n/2)

| n | naive c | calls | memo c | calls | iter c | lru c | C(n,k) |
|---|---|---|---|---|---|---|---|
| 10 | 0,0000 | 503 | 0,00001 | 51 | 0,00000 | 0,00001 | 252 |
| 15 | 0,0012 | 12869 | 0,00003 | 113 | 0,00000 | 0,00002 | 6435 |
| 18 | 0,0081 | 97239 | 0,00017 | 163 | 0,00000 | 0,00003 | 48620 |
| 20 | 0,0326 | 369511 | 0,00007 | 201 | 0,00001 | 0,00003 | 184756 |
| 22 | 0,1432 | 1.4e6 | 0,00007 | 243 | 0,00001 | 0,00006 | 705432 |
| 24 | — | — | 0,00047 | 289 | 0,00001 | 0,00017 | 2.7e6 |
| 28 | — | — | 0,00008 | 393 | 0,00000 | 0,00007 | 4.0e7 |

Наивная при n>22 не измерялась (время >> 1 с).

## 5.2. Г6: memo vs lru_cache

| n | Своя memo, с | Вызовов | lru_cache, с |
|---|---|---|---|
| 20 | 0,000042 | 201 | 0,000036 |
| 24 | 0,000058 | 289 | 0,000039 |
| 28 | 0,000079 | 393 | 0,000051 |

lru_cache чуть быстрее (C-реализация); семантика та же.

## 5.3. Максимальная сумма подотрезка (n=10 000)

| Показатель | Значение |
|---|---|
| Оптимальная сумма | 255 879 |
| Границы | [2 .. 9998], длина 9997 |
| Проверка sum | 255 879 |
| Время | ≈ 0,0015 с |

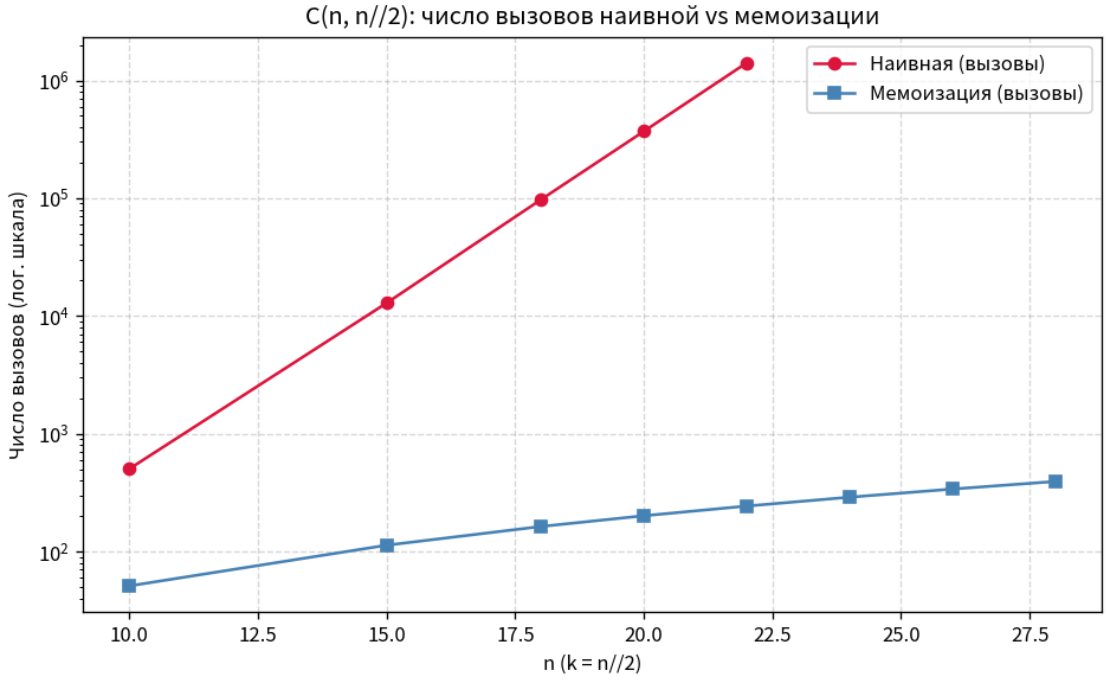



Рис. 1. Число вызовов: наивная vs мемоизация (лог. шкала)

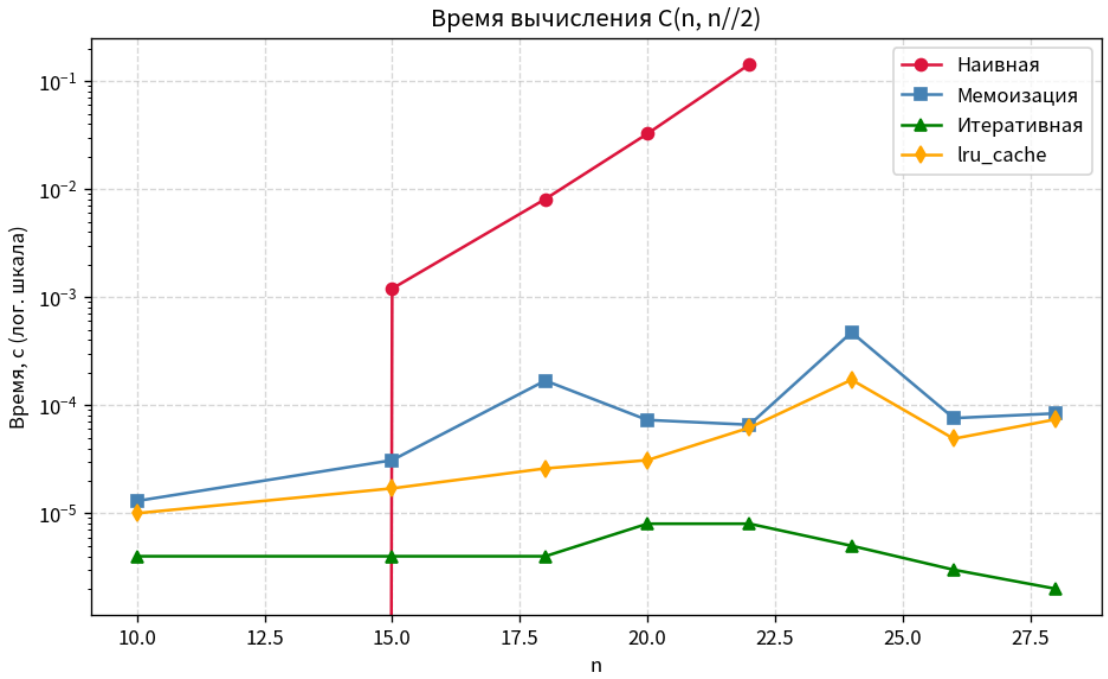



Рис. 2. Время вычисления C(n, n/2)

## 6. Выводы

1. Наивная рекурсия C(n,k) экспоненциальна: при n=22 уже >1,4 млн вызовов.
2. Мемоизация даёт O(nk) состояний; при n≤28 — сотни вызовов.
3. Итеративная формула — O(k) времени и O(1) памяти, самая быстрая.
4. Г6: lru_cache эквивалентен своей мемоизации и обычно быстрее.
5. Максимальная сумма подотрезка — Θ(n) с восстановлением границ (Кадане).
6. ДП устраняет повторные вычисления при перекрывающихся подзадачах.

Конец отчёта.